

Lab 02 — Commented answers

Each answer follows the same pattern: the answer itself, the mechanism behind it, a nuance or pitfall, and an example you can verify at the terminal.

Question 1 — Full names and short names

Answer.

Spelling	Full name	Podman
<code>nginx</code>	<code>docker.io/library/nginx:latest</code>	known alias → resolved without asking, with the message <code>Resolved "nginx" as an alias</code>
<code>bitnami/nginx</code>	<code>docker.io/bitnami/nginx:latest</code>	no alias → searched in <code>unqualified-search-registries</code> ; Ubuntu lists a single registry (<code>docker.io</code>), so it resolves; Fedora lists several, so Podman asks
<code>registry.mycompany.be:5000/base/nginx:1.25</code>	unchanged	full name: nothing to resolve

The rule is simple: if the first part (before the first `/`) contains a **dot** or a **colon** (a port), or reads `localhost`, it is a registry. Otherwise it is a namespace on the default registry.

Why. `mycompany.be` cannot be a Docker Hub username (dots are not allowed there), and a port only makes sense for a server. Docker applies this rule and then silently completes the name. Podman applies the same rule but refuses to complete blindly, because `nginx` on `docker.io` and `nginx` on `registry.internal` may be two different images.

Nuance. Despite its name, `bitnami/nginx` is **not** an official image — its namespace is `bitnami`, not `library`. And an image you build without naming a registry becomes `localhost/...`: that is a full name too, with `localhost` acting as a stand-in "registry".

Example.

```
podman pull nginx 2>&1 | head -2          # Resolved "nginx" as an alias ...
docker.io/library/nginx:latest
podman image inspect --format '{{.RepoTags}}' nginx
podman build -t api:1.0 . && podman images | grep api    # localhost/api 1.0
```

Question 2 — Same tag, different content

Answer. Someone **moved** the `2.3` tag in the meantime: they rebuilt an image and pushed it under the same name. Server A kept the old image (it never pulled again); server B received the new one. Comparing the digests proves it. Never reusing a published tag — and deploying by digest — would have prevented it.

Why. A tag is a mutable pointer on the registry side. A `pull` compares the remote digest with the local one and downloads only when they differ. Nothing warns you that a tag has moved.

Nuance. The move may well be unintentional: a pipeline that pushes `api:2.3` on every run of the release branch, or a `latest`. The base image can also move without your Dockerfile changing (FROM `eclipse-temurin:21-jre`): rebuilding "the same code" then yields a different image.

Example.

```
# on A and on B:
podman image inspect --format '{{.Digest}}' myapp/api:2.3
# different digests -> the tag moved. Correct deployment:
podman pull registry.internal/myapp/api@sha256:9d0d1f1e...
```

Question 3 — 62 GB shown, disk intact

Answer. No. `SIZE` reports each image's **virtual** size, shared layers included: common layers (JRE, Alpine, Debian) are counted in every image but stored once. `podman system df` reports the real usage. The files live in `~/.local/share/containers/storage` — inside the WSL distribution's virtual disk (`ext4.vhdx`), not in any Windows directory.

Why. The `overlay` driver stores each layer once, identified by its content, and images are nothing but lists of layers. Twelve Spring Boot images on the same JRE share its 180 MB.

Nuance. The WSL `vhdx` **grows** automatically but never **shrinks** on its own when you delete images. The space is freed on the Linux side, not on the Windows side, until you compact the disk (`wsl --shutdown`, then `Optimize-VHD` or `diskpart`). This surprises a lot of Windows users.

Example.

```
podman system df          # real SIZE and RECLAIMABLE
podman system df -v | head # SHARED SIZE column per image
podman info --format '{{.Store.GraphRoot}}' # /home/<you>/.local/share/containers/storage
```

Question 4 — The `rm` that removes nothing

Answer. They are wrong. `COPY` creates a layer that **contains** `credentials.json`. The `RUN ... rm` creates a later layer containing a *whiteout* that hides the file. The final image contains both layers: the file is still there, merely invisible from inside a container.

Why. Layers are immutable and additive. A layer cannot remove a file from an earlier layer; it can only hide it. Anyone with the image can run `podman save` on it, extract the `COPY` layer, and read the file.

Nuance. Combining the steps changes nothing here: even if the `rm` runs right after the copy, the `COPY` is still its own instruction with its own layer. Only a *build secret* (`RUN --mount=type=secret`) or a multi-stage build (lab 05) keeps the file out of the final image.

Example.

```
podman save --format oci-archive -o img.tar my-image:1.0
mkdir x && tar -xf img.tar -C x
for b in x/blobs/sha256/*; do tar -tf "$b" 2>/dev/null | grep -q credentials.json && echo
"present in $b"; done
```

Question 5 — 310 MB, 61 MB transferred

Answer. The registry already holds the unchanged layers (JRE, system, dependencies), so the `push` transfers only the new ones — the JAR and everything after it, 61 MB in total. If every push moved the full 310 MB, that would mean the image's early layers change on every build: a `COPY .` placed too early, or an instruction with variable content (a date, a version number) sitting before the heavy layers.

Why. Every layer has a digest. Before uploading a blob, the client asks the registry whether it already has it (`HEAD /v2/<repo>/blobs/<digest>`). Podman is less vocal about it than Docker (no `Layer already exists` line), but the transfer completes almost instantly.

Nuance. Whether layers are shared between *repositories* of the same registry depends on its implementation (Harbor and Docker Registry do it via *cross-repository mount*). And an "unchanged" layer must match bit for bit: a `RUN apt-get update` without pinned versions produces a different layer on every build.

Example.

```
podman push --tls-verify=false localhost:5000/base/demo:1.1 # instantaneous: blobs already present
podman history my-api:2.0 --format 'table {{.Size}}\t{{.CreatedBy}}' # spot the layer that changes
```

Question 6 — Two tags, one image, and a ghost

Answer. There are two distinct images: `f3a1b9c02d11` (carrying both tags `2.0` and `1.9`) and `8b2c74e91a03` (nameless). The `<none>` line is the *dangling* image: a tag (probably `2.0`) was rebuilt and moved, and the old image lost its name. `podman rmi api:1.9` removes **only the tag** (`Untagged: localhost/api:1.9`); the data stays, still referenced by `2.0`. And `localhost/` is the stand-in registry for any image built or tagged without a registry name.

Why. An `IMAGE ID` is the digest of the image configuration; two lines with the same ID are two names for one content. The data disappears only when its last name does.

Nuance. Don't confuse *dangling* (`<none>:<none>`, no tag at all) with *unused* (tagged, but with no container). A `podman image prune` without `-a` removes only the former.

Example.

```
podman rmi api:1.9 # Untagged: localhost/api:1.9 (no Deleted)
podman images --filter dangling=true -q # 8b2c74e91a03
podman rmi 8b2c74e91a03 # Deleted: ... this time the data goes
```

Question 7 — `exec format error`

Answer. The image was built for `linux/arm64` (Apple Silicon), and the server runs `linux/amd64`: the kernel cannot execute the binary. To unblock right away: rebuild with `--platform linux/amd64` (QEMU emulation — slow, but it works). To fix it for good: let **CI** build the images on `amd64` agents, or publish multi-architecture images (`podman manifest`).

Why. A multi-arch tag is a manifest list. At `build` time, the engine produces a manifest for the building machine's architecture. At `pull` time, the server looks for the `amd64` entry — it does not exist, so the server hands out `arm64`.

Nuance. Almost all *official* images are multi-arch, which hides the problem until the first in-house build. The error can also strike earlier and more quietly: running an `amd64` image on the Mac works (through emulation), just 5 to 10 times slower.

Example.

```
podman image inspect --format '{{.Architecture}}' registry.internal/api:1.4 # arm64
podman build --platform linux/amd64 -t registry.internal/api:1.4 .
```

Question 8 — `save` versus `export`, and the OCI format

Answer. `save` exports an **image**: layers, manifest, configuration, tags. `export` exports the file system of a **container**, flattened into a single tree, with no metadata. To carry a Spring Boot image to an isolated site, `save` is the right choice. With `export` you lose `ENTRYPOINT`, `CMD`, `ENV`, `EXPOSE`, `USER` and `WORKDIR` — the imported image no longer knows how to start — plus the layers themselves, so no more sharing and no more cache. `--format oci-archive` produces the same image in the standard OCI layout (`blobs/sha256/`, `index.json`) instead of Docker's historical format.

Why. `export` sees only the end result of assembling the layers, like a `tar` taken from inside the container. The configuration lives in the image, not in the file system.

Nuance. `export` does have legitimate uses: pulling a file system out for analysis, or building a "flat" image from a hand-configured container (bad practice, but documented). The `docker-archive` format remains the most common; use `oci-archive` whenever the recipient is not Docker (Kubernetes via `ctr`, skopeo...).

Example.

```
podman save --format oci-archive -o api.tar my-api:1.0
podman load -i api.tar # Loaded image: localhost/my-api:1.0
podman export c1 | podman import - flat:1 # Config.Cmd = null
```

Question 9 — The second `pull`

Answer. The engine asked the registry for the tag's **manifest**, compared its digest with the local image's digest, found them identical, and downloaded nothing else. A manifest is a few kilobytes; the network cost is one or two HTTP requests.

Why. The manifest and every layer are content-addressed, so the client knows exactly what it already has. A pull is always differential — in the extreme case, it transfers nothing at all.

Nuance. Podman never prints "Image is up to date"; it just prints the image identifier. "Under a second" also assumes a nearby registry: against Docker Hub, the request may add several seconds of latency without downloading a thing. And that request still counts against Docker Hub's quota (question 10).

Example.

```
time podman pull alpine # d529dd0c... - a few seconds of network, zero layers transferred
```

Question 10 — `toomanyrequests`

Answer. Docker Hub caps anonymous `pull`s per IP address (and per account for authenticated users). All the CI agents leave the network through the same public IP, so the whole company shares one quota — which is why failures look "random": they depend on how much has been pulled that hour. The two remedies: a **pull-through cache** (an internal registry that caches Docker Hub) and/or an **internal copy** of the base images in the company registry (`skopeo copy`), with Dockerfiles pointing at that registry.

Why. Every `pull` queries at least the manifest, even when the image is already local. A CI that rebuilds a hundred times a day burns through the quota quickly.

Nuance. Authenticating (`podman login docker.io`) raises the quota but does not remove it — and it puts a personal account into the CI. The internal copy brings an extra benefit: you control *what comes in* (scanning, validation), and you no longer depend on Docker Hub being up.

Example.

```
skopeo copy docker://docker.io/library/node:22-alpine docker://registry.internal/base/node:22-alpine
# then in the Dockerfile: FROM registry.internal/base/node:22-alpine
```

Question 11 — HTTP versus HTTPS

Answer. Docker treats `localhost` (and `127.0.0.0/8`) as an *insecure* registry by default: it accepts plain HTTP without comment. Podman makes no such exception: every registry must present a valid TLS certificate. There are two ways through: pass `--tls-verify=false` on the command, or add a `[[registry]] location = "localhost:5000" insecure = true` entry to `registries.conf`. The second option must never land in a version-controlled file or on a server: it silently disables verification for **every** use of that registry.

Why. Anyone on the network path can impersonate a registry that has no TLS (*man in the middle*) and serve a booby-trapped image. On `localhost` the risk is low — but Podman would rather have you say so explicitly than assume it.

Nuance. `--tls-verify=false` also disables **certificate** verification on a self-signed HTTPS registry. The better practice is to install the internal authority's certificate in `/etc/containers/certs.d/<registry>/ca.crt`.

Example.

```
podman push --tls-verify=false localhost:5000/base/demo:1.0 # explicit, visible in the history
# OR, for a development workstation only:
printf '[[registry]]\nlocation = "localhost:5000"\ninsecure = true\n' >>
~/.config/containers/registries.conf
```

Question 12 — `image is in use by a container`

Answer. A container created from that image still exists — possibly stopped — and the image is its base layer. The engine refuses because removing the image would break that container. The clean way: list the container (`podman ps -a --filter ancestor=...`), remove it once you no longer need its state, then run `rmi`. `podman rmi -f` removes the tag, the image **and** the dependent

container without asking: you lose the logs, the writable layer and any chance to inspect the container — all to save ten seconds.

Why. A container is an image plus a writable layer. Without the image, the writable layer is meaningless.

Nuance. Podman's message mentions "external containers": containers created by Buildah or another tool sharing the same storage, which `podman ps` does not show. `podman ps -a --external` reveals them. Docker refuses in the same situation, but its message spells out the container ID.

Example.

```
podman ps -a --filter ancestor=my-api:1.0 --format '{{.Names}} {{.Status}}'
podman logs <container> > incident.log      # we save what must be saved
podman rm <container> && podman rmi my-api:1.0
```

Question 13 — The 0B layers

Answer. They come from **metadata** instructions — `ENV`, `CMD`, `ENTRYPOINT`, `EXPOSE`, `LABEL`, `USER`, `WORKDIR` — which change the image configuration without touching the file system. They show up in the history because every instruction leaves a trace, even an empty one. The 180 MB layer (a `RUN apt-get install`, a `COPY` of a JRE) is the only one that matters: `podman history` points straight at the line worth optimising.

Why. An image is a list of instructions, some of which carry a blob of files. All the weight sits in the blobs.

Nuance. A non-empty layer can hide a deletion: a `RUN rm -rf /var/lib/apt/lists/*` in its own `RUN` weighs almost 0B yet reclaims nothing. `history` shows what each step costs; `podman image tree` additionally shows which layers come from the base image.

Example.

```
podman history nginx:alpine --format 'table {{.Size}}\t{{.CreatedBy}}' # 12 lines at 0B, one
at 50.7MB
podman image tree nginx:alpine
```

Question 14 — Three tagging strategies

Answer. (a) An overwritten `latest` makes rollback impossible — the old image no longer has a name — and diagnosis impossible, since nobody can tell which version was running. (b) `1.4.2` allows an immediate rollback to `1.4.1` and supports diagnosis as long as the tag stays immutable; but two builds of `1.4.2` (say, after a quick fix) can coexist with nothing to tell them apart. (c) `1.4.2-b318-a9f3c21` makes every image unique: you can trace back to the exact commit and build, and roll back to any earlier build. The price is an unwieldy name and a retention policy to manage.

Why. Deployment and diagnosis both need a **one-to-one** match between a name and a content. Only (c) guarantees that by construction; (b) relies on discipline; (a) rules it out.

Nuance. The three often coexist: CI pushes (c); a (b) tag is *added* to the same image once it is validated; `latest` exists purely for developer convenience and never appears in a deployment manifest. And the deployment itself pins the digest.

Example.

```
podman tag registry.internal/api:1.4.2-b318-a9f3c21 registry.internal/api:1.4.2    # same image,  
second name  
podman image inspect --format '{{.Digest}}' registry.internal/api:1.4.2          # what is  
actually deployed
```